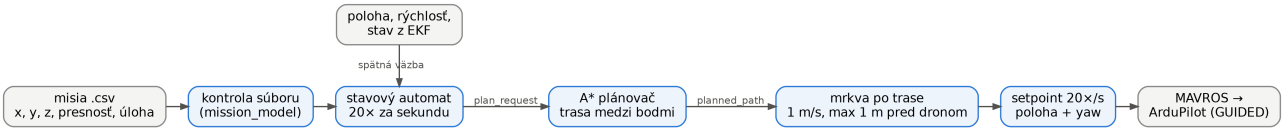
A1.2 — Indoor misia v hangári (waypointy, úlohy, A*)

Dron v simulácii (ArduCopter SITL + Gazebo, cez MAVROS) preletí misiu zo súboru: vzlietne, ku každému bodu si nechá naplánovať trasu algoritmom A* z A1.1, preletí ju s nosom v smere letu, vykoná úlohu bodu (yaw, pristátie, pristátie a znovu vzlet) a na konci pristane a odzbrojí sa. Všetko riadi jeden uzol `mission_executor_node` so stavovým automatom.

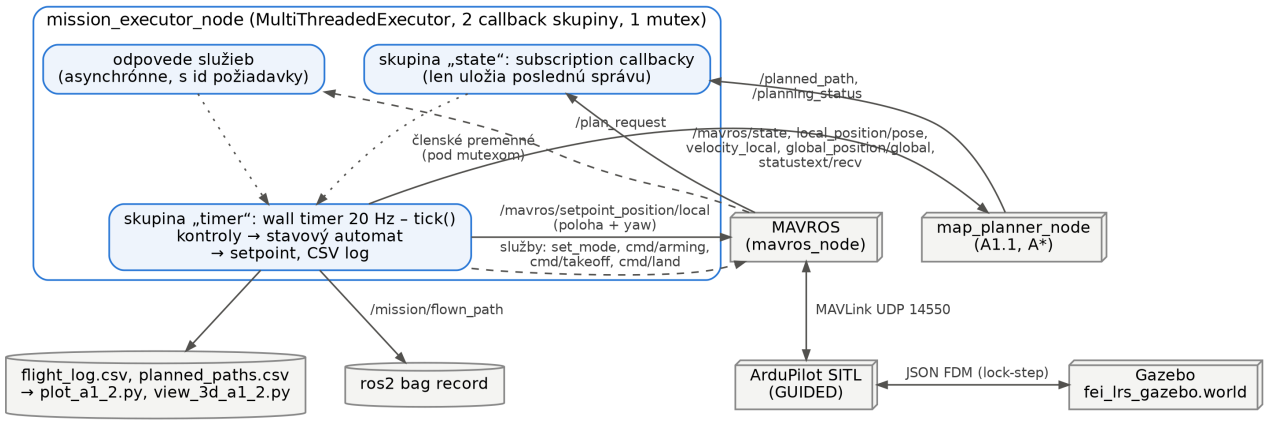
Spustenie: `./run_a1_2.sh [misia.csv]` (bez argumentu `missions/hangar_example.csv`). Skript preloží balík, otvorí Gazebo, SITL a MAVROS v troch oknách, spustí plánovač a executor, nahrá rosbag a po skončení vykreslí grafy a 3D pohľad. Prepínače: `A1_2_STOP_SIM=1` (na konci zavrie simulátor), `A1_2_SKIP_BUILD=1`, `A1_2_RVIZ=1`. **Záznam misie:** `results/hangar_example_20261018/`.

Diagramy

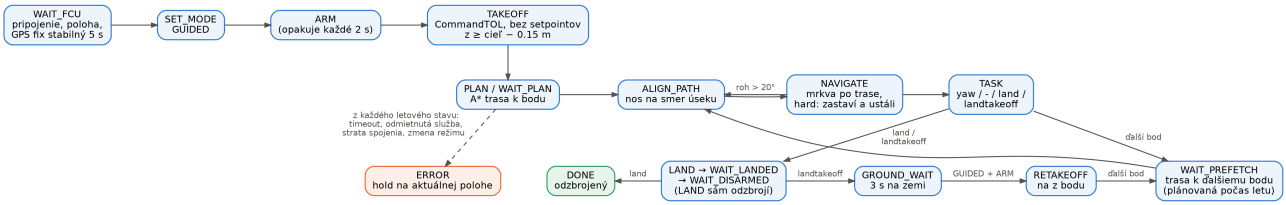
Tok dát: misia obsahuje len cieľové body. Trasu medzi nimi počíta A* a dron ju sleduje pomocou „mrkvy“.



ROS diagram: jeden uzol, `MultiThreadedExecutor` s dvomi skupinami callbackov (vzor `mt_executor_demo`). Callbacky len uložia poslednú správu, všetko rozhoduje časovač 20× za sekundu. Zdieľané dáta chráni jeden mutex.



Stavový automat: prerušovaná šípka (ERROR) platí z každého letového stavu.



Rozhodnutia

Misia zo súboru

Čo: cesta k misii je ROS parameter `mission_file`. Formát `x,y,z,presnosť,úloha` (`hard`/`soft`; úloha `-`, `takeoff`, `land`, `landtakeoff`, `yaw<uhol>`, napr. `yaw-45`). Riadky s `#` sú komentáre. Kontrola je prísna: zlý počet polí, nečíslo alebo neznáma úloha zastaví uzol ešte na zemi a vypíše číslo riadku.

Prečo: na obhajobe dostaneme neznámy súbor. Preklep musí skončiť jasnou chybou, nie čudným letom.

Súradnice a konvencia yaw

Čo	Rámec
misia, plánovač, grafy (`map`)	rámec mapy a sveta Gazeba: x = sever, y = západ, z hore
MAVROS lokálne	ENU (x = východ, y = sever), otočené o 90° voči `map`
ArduPilot vnútri	NED, prevod ENU ↔ NED robí MAVROS za nás

Prevod `map ↔ local` je otočenie o 90° okolo z (`map_to_local_yaw_offset_deg`). Overili sme ho: dron štartuje v Gazebe na (12.99, 7.00) a MAVROS hlási lokálne (−7.00, 12.99). Vnútri executora je všetko v rámci `map`, prevádzame len na hranici s MAVROS.

Yaw v úlohe `yaw<uhol>`: absolútny smer v rámci mapy, v stupňoch, **kladne proti smeru hodinových ručičiek**, 0° = os +x mapy. Nie je relatívny k aktuálnemu natočeniu. `yaw90` = nos na +y, `yaw-45` = 45° v smere hodinových ručičiek od +x.

Pripravenosť pred armovaním

Čo: v stave `WAIT_FCU` čakáme, kým je MAVROS pripojený, prichádza lokálna poloha a GPS má fix aspoň 5 s bez prerušenia (alebo EKF ohlási „is using GPS“). Všetky textové správy z ArduPilota (`PreArm: ...`, `Arm: Gyros inconsistent`) vypisujeme do nášho logu.

Prečo: bez polohy ArduPilot odmietne GUIDED aj armovanie. Pôvodný skript čakal na tieto správy v logu SITL, ale MAVProxy ich zobrazuje len vo svojom okne, takže čakanie vždy skončilo až po 240 s timeoute. Teraz štart trvá ~1.5 min namiesto ~5 min.

Plusy / mínusy: + dôvod odmietnutia je vidieť priamo v našom logu. − Pri pomalej simulácii ArduPilot ešte ~20–40 s hlási „Gyros inconsistent“. Executor armovanie opakuje každé 2 s.

GUIDED, armovanie a služby

Čo: všetky služby (set_mode, cmd/arming, cmd/takeoff, cmd/land) voláme asynchrónne a kontrolujeme odpoveď. Ďalej sa ide až vtedy, keď `/mavros/state` potvrdí skutočný stav (režim GUIDED, armed). Každá požiadavka má číslo; neskorú odpoveď zo starého stavu ignorujeme. Bez odpovede do 5 s → ERROR.

Prečo: `mode_sent = true` znamená len „odoslané“, ArduPilot môže režim aj tak odmietnuť. Pôvodný kód posielal GUIDED každých 50 ms; teraz najviac raz za 2 s.

Vzlet bez setpointov

Čo: po armovaní pošleme `NAV_TAKEOFF` na výšku prvého bodu a **neposielame žiadne polohové setpointy**, kým dron nevystúpi do 0.15 m od cieľovej výšky. Ak nestúpa 12 s, príkaz pošleme znova; ak sa na zemi odzbrojí, zopakujeme GUIDED → ARM → TAKEOFF (najviac 3×).

Prečo: v GUIDED režime polohový setpoint zruší rozbehnutý vzlet. Riadenie polohy na zemi nezdvihne motory a po `DISARM_DELAY` (10 s) sa dron sám odzbrojí. Presne to bola pôvodná chyba („zaarmuje, odzbrojí sa a nevzlietne“).

Plánovanie trasy a predplánovanie

Čo: pre prvý úsek pošleme `plan_request` [aktuálna poloha, bod]. Počas letu k bodu i už žiadame trasu i → i+1 (predplánovanie). Trasu odmietneme, ak jej koniec je od bodu misie ďalej, než je tolerancia bodu (plánovač mohol cieľ posunúť).

Prečo: po príchode k bodu je ďalšia trasa hneď pripravená a dron nevisí, kým A* počíta.

Sledovanie trasy „mrkvou“

Čo: dron neposielame rovno na koniec úseku. Setpoint (mrkva) sa posúva po priamke úseku rýchlosťou 1 m/s a nikdy nie je ďalej ako 1 m pred dronom. Medziľahlé body trasy sa berú v okruhu 0.45 m bez zastavenia.

Prečo: keď sme posielali len koniec úseku, ArduPilot letel s inými limitmi rýchlosti vodorovne a zvislo. Dron opúšťal priamku (až 0.56 m), napríklad klesal skôr. Trasa A* je bezkolízna len na priamke, čo je dôležité hlavne pri lete ponad regál. S mrkvou dron drží priamku aj pri stúpaní z 1 m na 4.8 m.

Plusy / mínusy: + dron ostane na naplánovanej trase. – Rýchlosť je obmedzená na 1 m/s (pri pomalej simulácii sa škáluje).

Smer nosa a otáčanie bez skokov

Čo: medzi bodmi letí dron nosom v smere úseku (`atan2(Δy, Δx)`). Pred trasou a v rohoch ostrejších ako 20° sa najprv otočí na mieste (`ALIGN_PATH`, do 5° počas 0.5 s), miernejšie zákruty letí plynule. Príkaz yaw držíme spojitý (unwrap): z 170° na −170° sa dron otočí o 20°, nie o 340°. Preto log niekedy ukazuje napr. „−540°“, čo je ten istý smer ako 180°.

Prečo: zadanie žiada let v smere pohybu a žiadne otočky o celé kolo.

Presnosť hard a soft

	Tolerancia	Kedy je bod splnený
`hard`	0.15 m	do 0.15 m **a** rýchlosť ≤ 0.2 m/s nepretržite 1 s (aspoň 2 rôzne správy rýchlosti)
`soft`	0.55 m	do 0.55 m, bez zastavenia

Prečo: 0.15 m je v odporúčanom rozsahu 10–20 cm a v SITL sa spoľahlivo dosahuje (namerané 2.9–9.7 cm). Pri 0.55 m dron prejde bodom plynule a už sa točí na ďalší úsek. Tolerancie nie sú bezpečnostná rezerva voči prekážkam, tú zabezpečuje safety_radius plánovača. Počítame rôzne správy rýchlosti, aby jedna stará správa videná 20× nevytvorila falošnú „ustálenú sekundu“.

Úloha yaw

Čo: na bode sa cieľový yaw nastaví na zadaný uhol (absolútne, pozri konvenciu vyššie). Úloha je splnená, keď je nos do 5° od cieľa nepretržite 0.5 s. Poloha sa medzitým drží.

Prečo: uhol posielame ako orientáciu polohového setpointu. Ten je v GUIDED zapamätaný, takže netreba posielať rýchlosti, ktorým by vypršal `GUID_TIMEOUT`.

Pristátie a landtakeoff

Čo: `cmd/land` prepne ArduPilot do LAND. Po dotyku (z ≤ 0.15 m, rýchlosť ≤ 0.15 m/s) čakáme, kým LAND **sám odzbrojí** dron. Vlastný príkaz na odzbrojenie pošleme až po 5 s a pri odmietnutí ho opakujeme. Pri `landtakeoff` potom 3 s počkáme na zemi, znova prepneme na GUIDED (dron je v LAND), zaarmujeme a vzlietneme na výšku bodu.

Prečo: ArduPilot prijme odzbrojenie, až keď jeho vlastný detektor pristátia potvrdí dotyk, asi sekundu po našej podmienke. Pôvodný kód bral odmietnutie ako chybu a misia skončila na bode landtakeoff.

Vlákna a časovač

Čo: `MultiThreadedExecutor`, skupina `state` (všetky subscription), skupina `timer` (časovač 20 Hz). Všetky zdieľané premenné chráni jeden mutex, ktorý drží každý callback aj celý `tick()`. Služby sú asynchrónne, takže `tick()` nikdy nečaká s držaným mutexom.

Prečo: podľa zadania (vzor `mt_executor_demo`). Príval správ nikdy neoneskorí riadiaci cyklus a rozhodnutia sa robia nad konzistentnou kópiou dát.

Plusy / mínusy: + riadenie beží presne 20× za sekundu. – Mutex je nutný (pri jednom vlákne by nebol).

Chyby a bezpečnosť

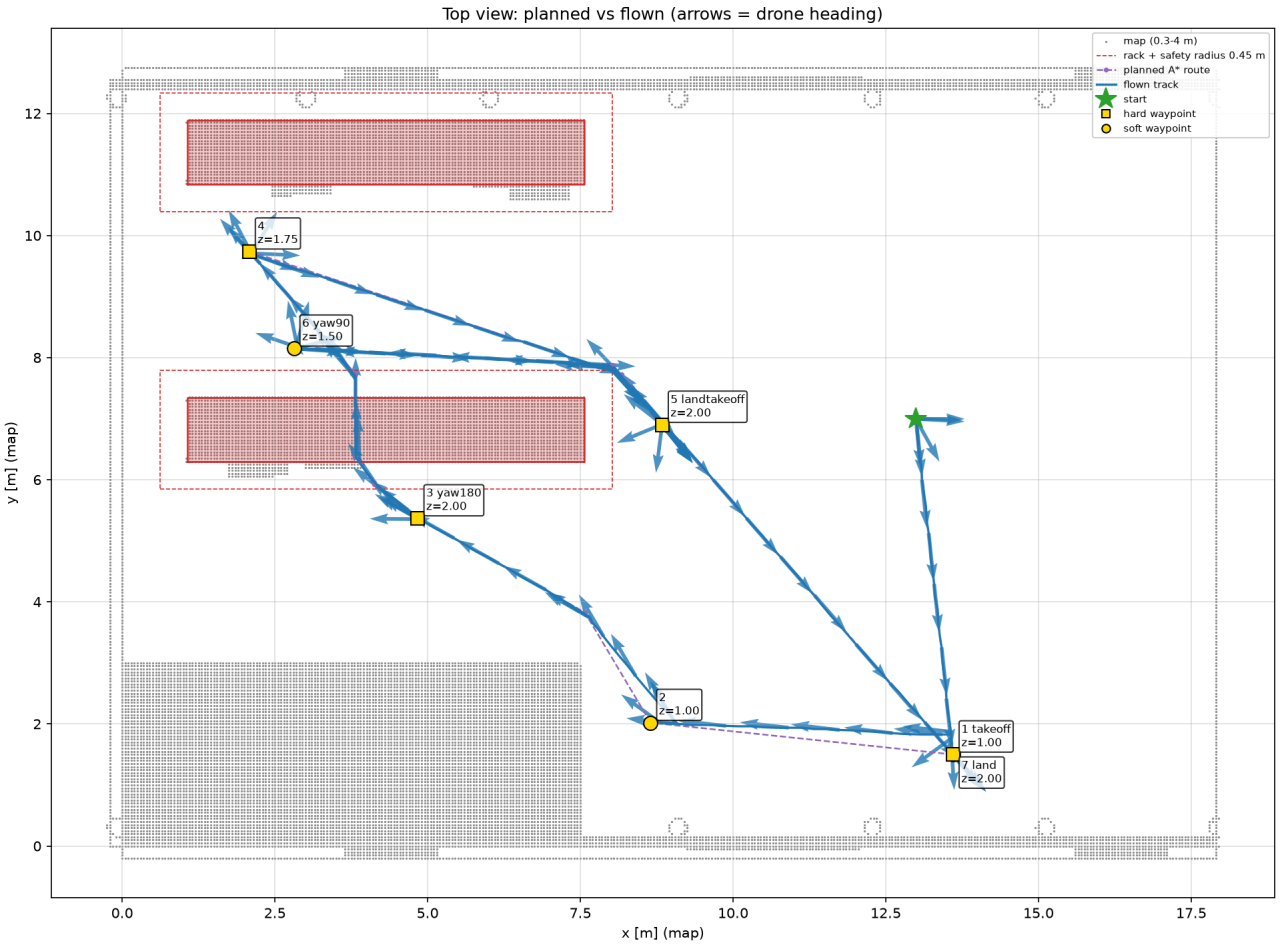
- **ERROR:** timeout stavu, odmietnutá služba, chýbajúca odpoveď, strata spojenia, poloha staršia ako 5 s, nečakané odzbrojenie → dron dostane ako cieľ svoju aktuálnu polohu a visí.
- **Nebojujeme s pilotom:** keď počas letu niekto prepne režim mimo GUIDED, misia sa zastaví.
- **Pomalá simulácia:** na našom PC beží Gazebo na 0.2–0.3× reálneho času. Skript zmeria pomer a časové limity executora vynásobí (napr. ×4), rýchlosť mrkvy vydelí.
- **Kontrola kolízií po lete:** `plot_a1_2.py` pre každú zaznamenanú polohu spočíta vzdialenosť k najbližšiemu bodu mapy (steny, stĺpy, nosníky strechy; podlaha nie) alebo regálu a nahlási minimum voči polomeru drona 0.33 m.

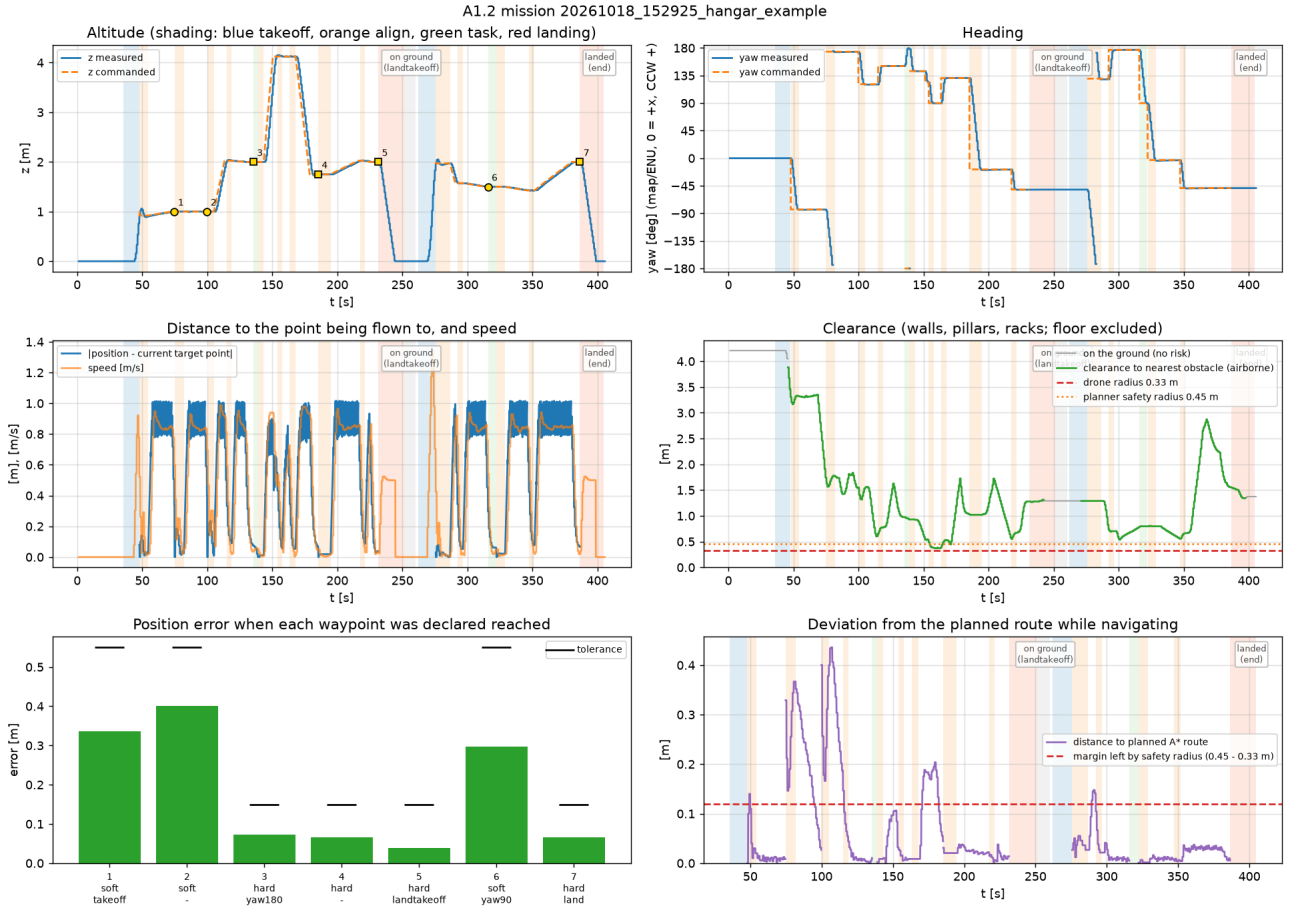
Výsledky (simulácia)

Záznam je v `results/hangar_example_20261018/` (rosbag, CSV log, trasy, grafy). Všetky misie prebehli bez zásahu od štartu po odzbrojenie.

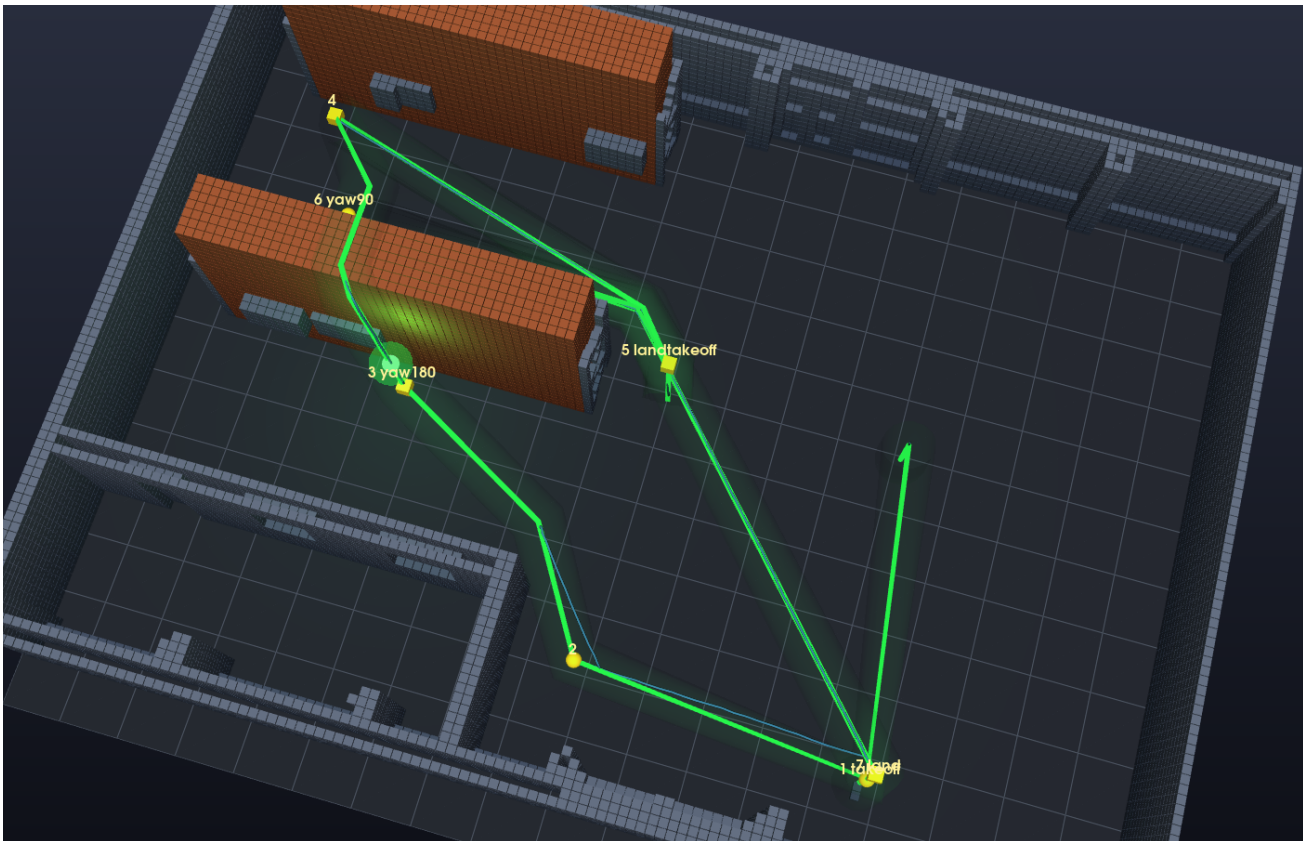
Misia	Body	Výsledok	Chyba hard bodov	Chyba soft bodov	Najmenší odstup
`hangar_example.csv` (záznam)	7	DONE, odzbrojený	3.9–7.3 cm	0.30–0.40 m	0.37 m
`test_east_aisle.csv`	8	DONE, odzbrojený	2.9–7.0 cm	0.07–0.34 m	0.61 m
`test_over_racks.csv`	7	DONE, odzbrojený	5.0–7.6 cm	0.30–0.35 m	0.38 m

`hangar_example` po bodoch: takeoff 0.335 m (soft), – 0.400 m (soft), yaw180 0.073 m, – 0.066 m, landtakeoff 0.039 m, yaw90 0.297 m (soft), land 0.066 m. Let trval 404 s pri rýchlosti simulácie ~0.3. A* viedol úsek k bodu 4 ponad regál (z ≈ 4.1 m) a dron ho sledoval.





Commanded lines are blank during takeoff and landing on purpose: ArduCopter flies NAV_TAKEOFF / LAND itself and the executor sends no position setpoints then (a setpoint would cancel the takeoff). Route deviation is only defined while navigating.



Prerušené čiary v grafoch nie sú chýbajúce dáta: počas vzletu, pristátia a na zemi executor zámerne neposiela setpoint, takže „požadovaná“ výška a smer v tom čase neexistujú.

Čo nefunguje dokonale

- **Malá rezerva nad regálom:** najmenší odstup 0.37–0.38 m, len 4–5 cm nad polomerom drona. Príčinou je zaokrúhlenie voxelov v A1.1 a v `test_over_racks` aj pokles drona ~0.15 m pod trasu pri klesaní. Pomohol by `safety_radius` ≈ 0.53 m, ale potom sa nedajú použiť body bližšie ako ~0.55 m od prekážok.
- **Po soft bode** začína ďalší úsek v strede bodu, takže dron je prvý meter až 0.4 m od nakreslenej trasy (v otvorenom priestore, odstup tam bol > 1.5 m).
- **Krátke zastavenia v ostrých rohoch** (> 20°) kvôli otočeniu nosa.
- **Pomalá simulácia:** pri 0.3× reálneho času trvá štart ~1.5 min a misia ~7 min.
- **Gazebo nesmie byť pozastavené:** SITL beží s Gazebom v kroku (lock-step). Kliknutie na ⏸ zastaví ArduPilot, MAVROS stratí spojenie a misia skončí v ERROR.